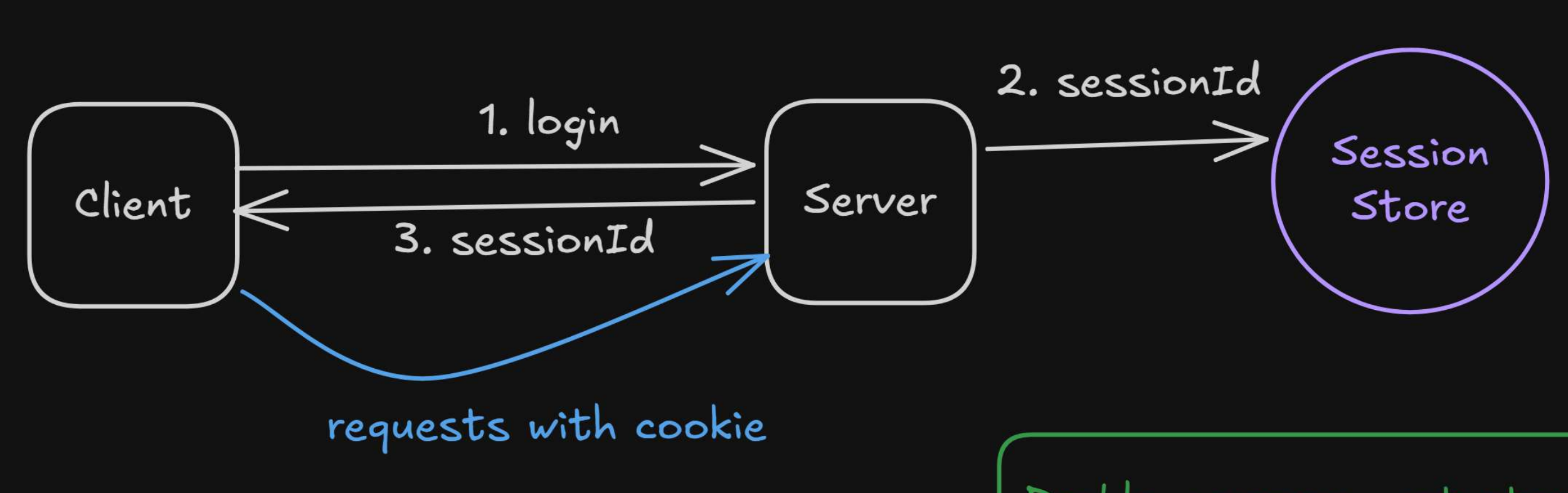
JWT

- JSON Web Token

The Real Problem (Why JWT Exists)

Before JWT, we had session-based authentication.

How login worked (old way)



Problem: server must store state

User logs in
Server creates a session (in DB/Redis)
Server sends session ID in cookie
Every request → server looks up session

Problems with Sessions

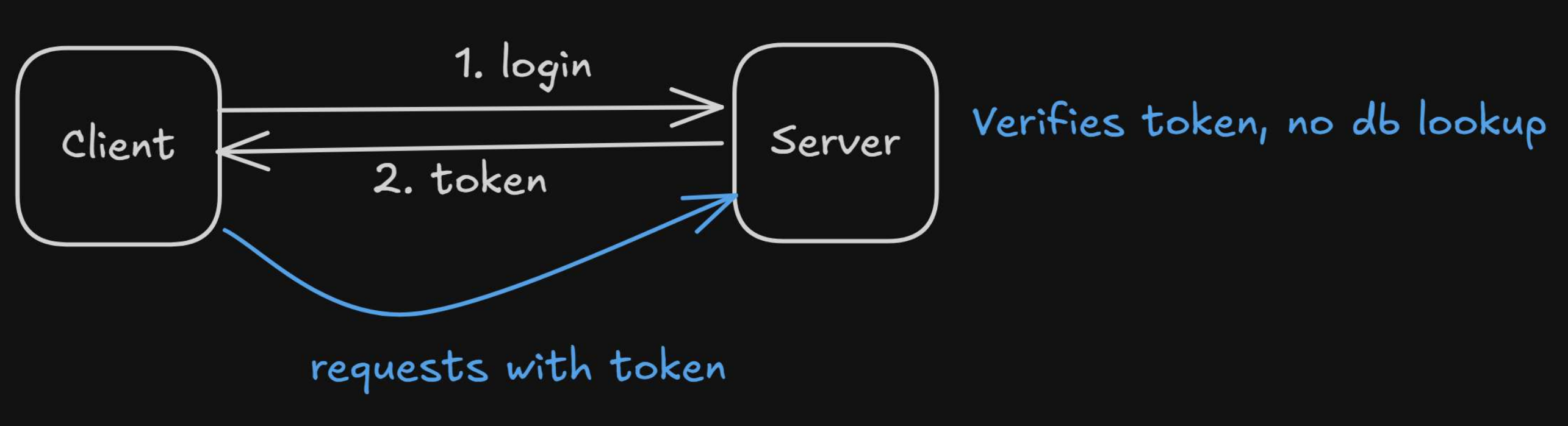
1. Scaling issue
   If you have multiple servers:
   -> Session must be shared (Redis, DB)
   -> Adds complexity
2. Performance overhead
   Every request → DB/Redis lookup
3. Microservices problem
   Each service needs session access
4. Mobile / APIs
   Cookies not always ideal

So we needed:

--> A way to authenticate users without storing session on server

Enter JWT (The Idea)

JWT solves this by making auth:
Stateless



Instead of storing session:

Server gives user a token
Token contains user identity
Server just verifies token → no DB lookup

What is JWT (Fundamental Concept)

JWT (JSON Web Token) is a compact, URL-safe token format
used to securely transmit data between parties

Format:

HEADER.PAYLOAD.SIGNATURE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmF
tZSI6IkpvaG4gRG9lIiwiYWRtaW4iOnRydWUsImlhdCI6MTUxNjIzOTAyMn0.KMUFsID
TnFmyG3nMiGM6H9FNFUROf3wh7SmqJp-QV30
```

Key idea:

It's self-contained → contains user data (claims)
It's stateless → server doesn't store session
It's signed (and optionally encrypted)

Structure of JWT

1. Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

alg → algorithm used (HS256, RS256, etc.)
typ → always JWT

The algorithm in JWT defines how the token is signed and verified,
including the hashing method and key type.
It ensures both parties use the same cryptographic process.
Choosing the correct algorithm is critical for security, and
servers must not trust the algorithm specified in the token header
to avoid attacks like algorithm confusion.

RS256 → use public key
HS256 → use secret

Prefer RS256 in production

2. Payload (Claims)

```
{
  "userId": "123",
  "role": "admin",
  "exp": 1716239022
}
```

Contains:

user identity
permissions
expiry

This is visible to anyone

JWT payload is not encrypted and can be decoded by anyone,
so fields like userId are visible.
However, this is safe because the token is signed,
not encrypted—meaning the data cannot be tampered with.
Sensitive information should never be stored in the payload.

3. Signature

Generated using:

```
Hash(
  base64Url(header) + "." + base64Url(payload),
  secret
)
```

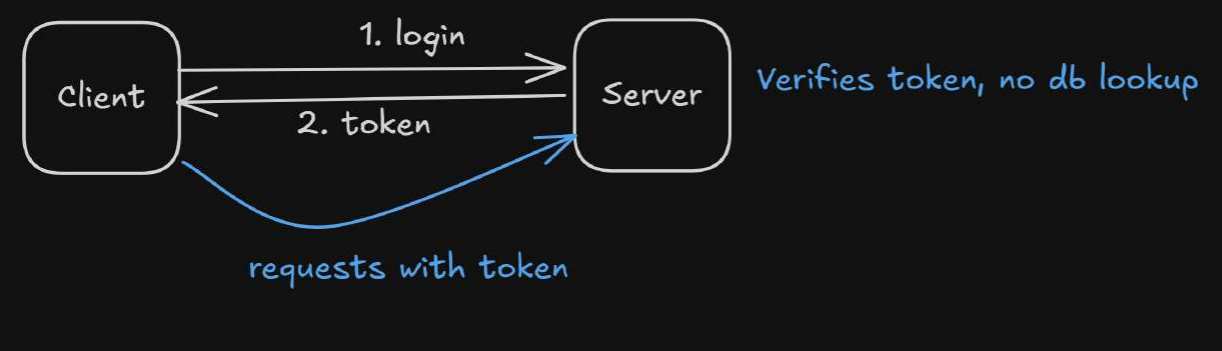
Ensures:

Token not tampered
Comes from server

Types of JWT

1. JWS (Signed JWT)
   Most common
   Only signed, not encrypted
2. JWE (Encrypted JWT)
   Payload encrypted
   Used when sensitive data needed

How JWT Authentication Works (Flow)



Step 1: Login
User sends:
email + password

Step 2: Server verifies user

Step 3: Server creates JWT
Adds userId, role, expiry
Signs it with secret

Step 4: Send token to client

Step 5: Client stores token
localStorage / cookie

Step 6: Every request

Authorization: Bearer <token>

Step 7: Server verifies token
Check signature
Check expiry

If valid → allow access

Key Breakthrough (Why JWT is Powerful)

Server does NOT need:

DB lookup
Session storage

-> Because:

Token already contains everything

Trade-offs

JWT is not magic.

-> Problem 1: Cannot logout easily
Token valid till expiry

-> Problem 2: Token theft = full access
No server control

-> Problem 3: Large token size
Sent in every request

Access Token + Refresh Token (Modern Design)

Industry standard (2026)

-> Access Token
Short life (5–15 min)

-> Refresh Token
Long life
Used to get new access token

Flow:

Login → get both tokens
Access token expires
Use refresh token → get new access token

JWT vs Sessions (Core Differences)

Feature	JWT	Session
Storage	Client	Server
State	Stateless	Stateful
Scalability	High	Lower
Revocation	Hard	Easy
Size	Larger	Small

DEMO

How Server Verifies JWT (Step-by-Step)

When a request comes:

Authorization: Bearer <JWT>

Step 1: Split the Token

JWT:
HEADER.PAYLOAD.SIGNATURE

Server separates:

header
payload
signature

Step 2: Decode Header + Payload

Base64 decode

Read:
algorithm (alg)
claims (userId, exp, etc.)

Important:

This step does NOT mean it's trusted yet

Step 3: Recreate Signature

Server does:

expectedSignature = hash(header + payload + secret)

-> Same algorithm used (e.g., HS256)

Step 4: Compare Signatures

```
if (expectedSignature == receivedSignature)
    → valid
else
    → reject
```

Step 5: Validate Claims

Even if signature is valid:

Check expiry (exp)
Check issuer (iss)
Check audience (aud)

Final Logic

-> Server trusts token ONLY IF:

Signature matches
Token not expired
Claims valid

Two Models for Verification

-> 1. Symmetric (HS256)
Same secret used for:
signing
verification

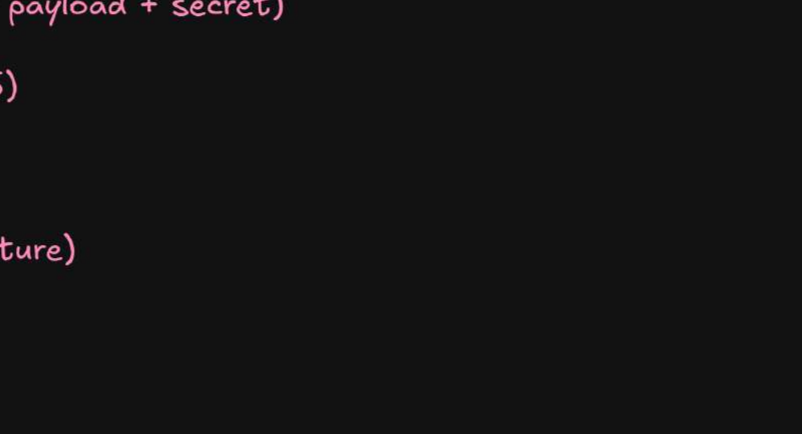
-> Example:

secret = "mySuperSecretKey"

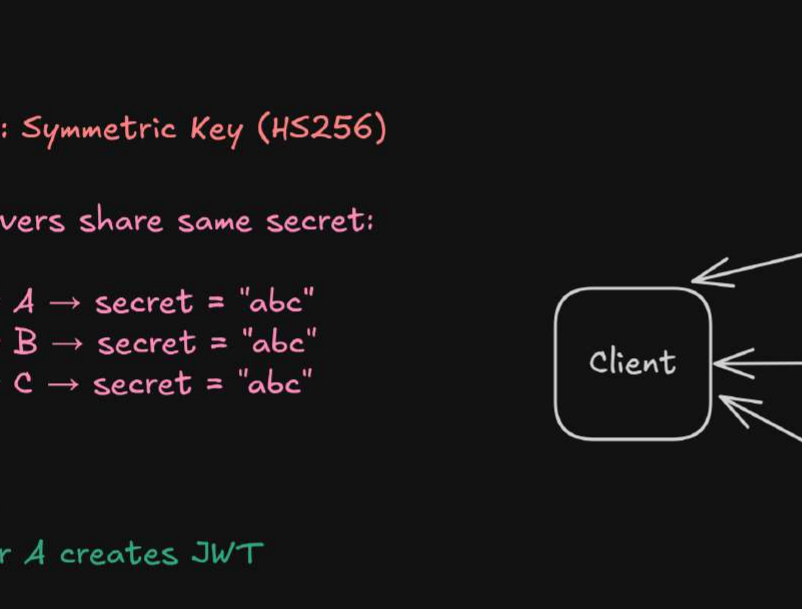
All servers must know this.

-> 2. Asymmetric (RS256)
Private key → sign
Public key → verify

-> Better for distributed systems



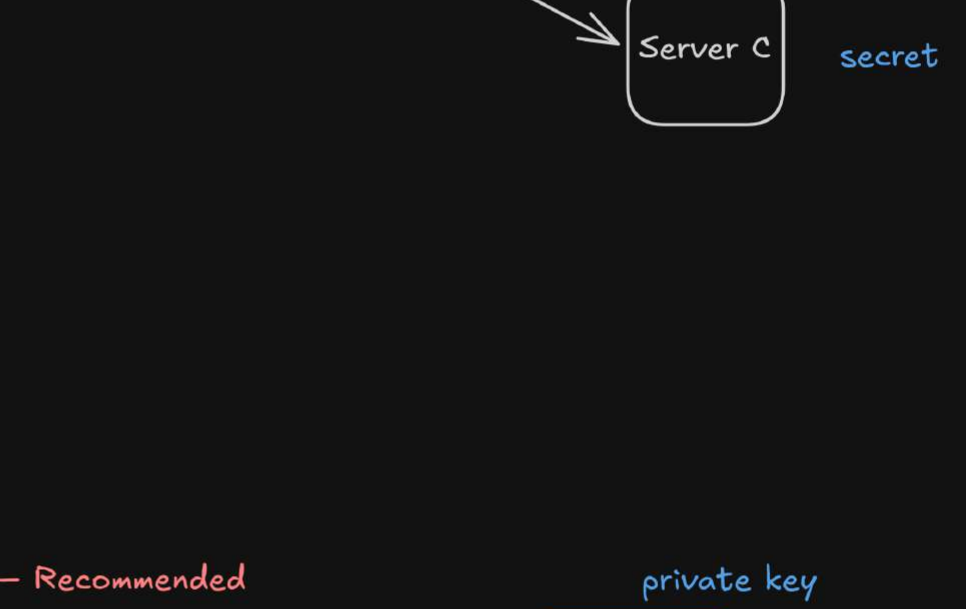
What if we have multiple server instances



Case 1: Symmetric Key (HS256)

All servers share same secret:

Server A → secret = "abc"
Server B → secret = "abc"
Server C → secret = "abc"



Flow:
Server A creates JWT

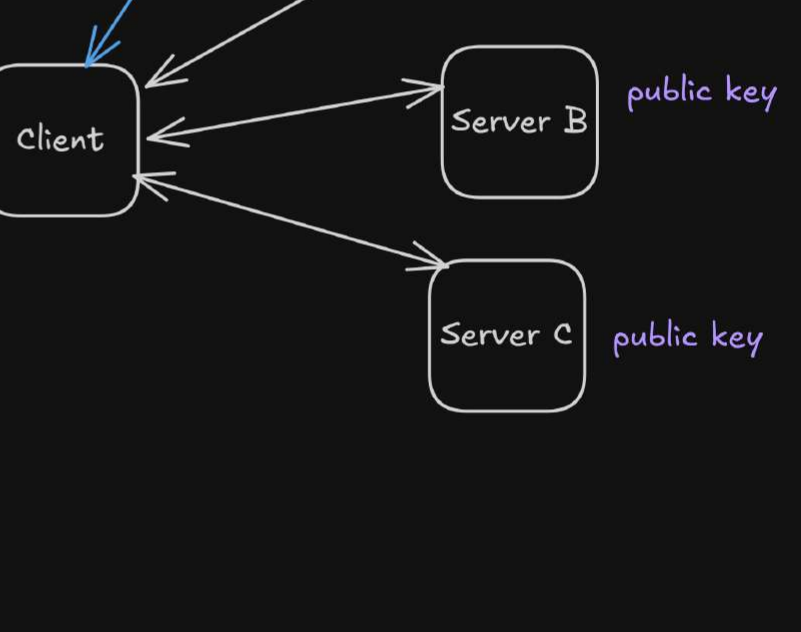
Client sends request to Server B

Server B verifies using same secret

-> Works perfectly
Problem:
Secret must be shared securely
Harder in large systems

Case 2: Asymmetric Keys (RS256) — Recommended

Setup:
Auth server → has private key
All services → have public key



Flow:
Auth server signs token using private key

Any service verifies using public key

No secret sharing needed
More secure

Why JWT is Perfect for Microservices

In microservices:

10–50 services
Requests go anywhere

-> With sessions:

Need central Redis

-> With JWT:

Each service verifies locally

JWT vs Session in Multi-Server Setup

Feature	Session	JWT
Multi-server	Needs Redis	Works natively
DB lookup	Required	Not needed
Latency	Higher	Lower

Real Production Setup

-> Auth Service
Issues JWT

-> API Gateway
Verifies JWT

-> Microservices
Trust JWT (or re-verify)

Critical Mistakes Developers Make

1. Not verifying signature

→ full security bypass

2. Trusting payload directly

→ attacker can modify

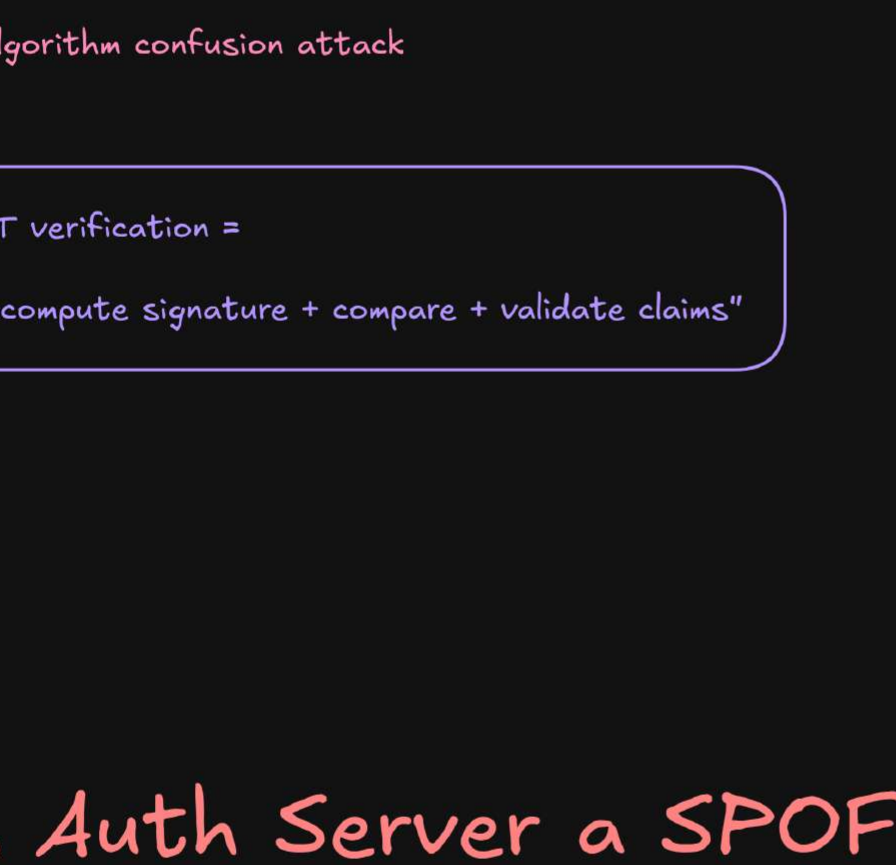
3. Accepting any algorithm

→ algorithm confusion attack

JWT verification =

"Recompute signature + compare + validate claims"

Is Auth Server a SPOF?



In a JWT-based system, the auth server is not a single point of failure for request processing because token verification is done locally by each service.
Even if the auth server goes down, existing users continue to function.
To avoid SPOF for login and token issuance,
multiple stateless auth server instances are deployed behind a load balancer.

Security Mechanisms

For JWT

Verify signature
Validate expiry
Use RS256 (public/private keys)

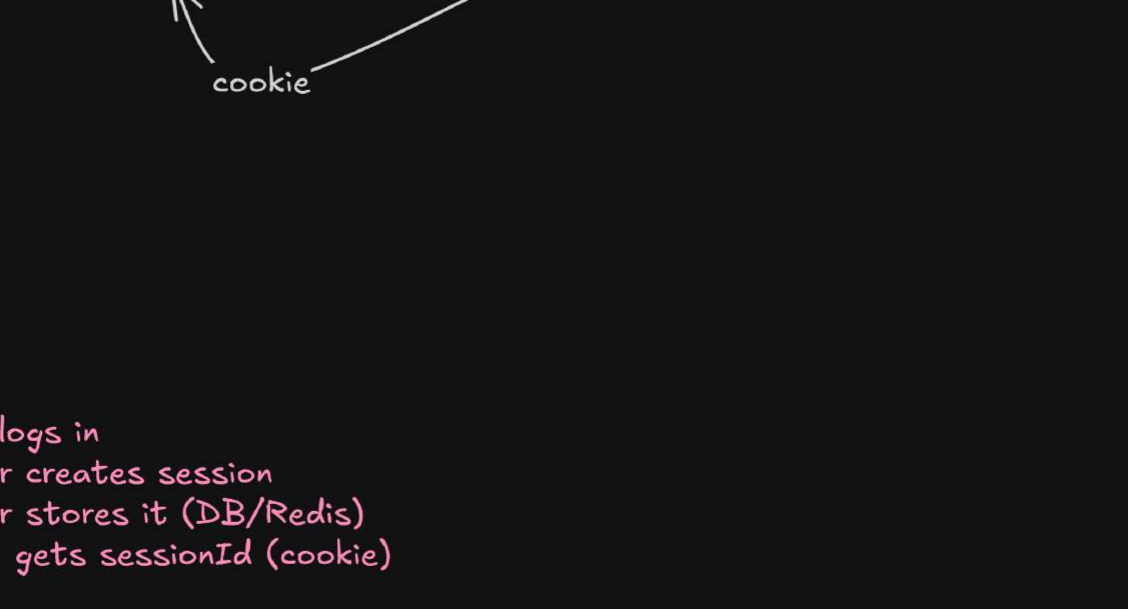
For Refresh Tokens

Hash before storing
Rotate every use
Detect reuse
Set expiry
Store in HTTP-only cookies

JWT vs Session

Session = server remembers you
JWT = you prove yourself every time

Session-Based Authentication

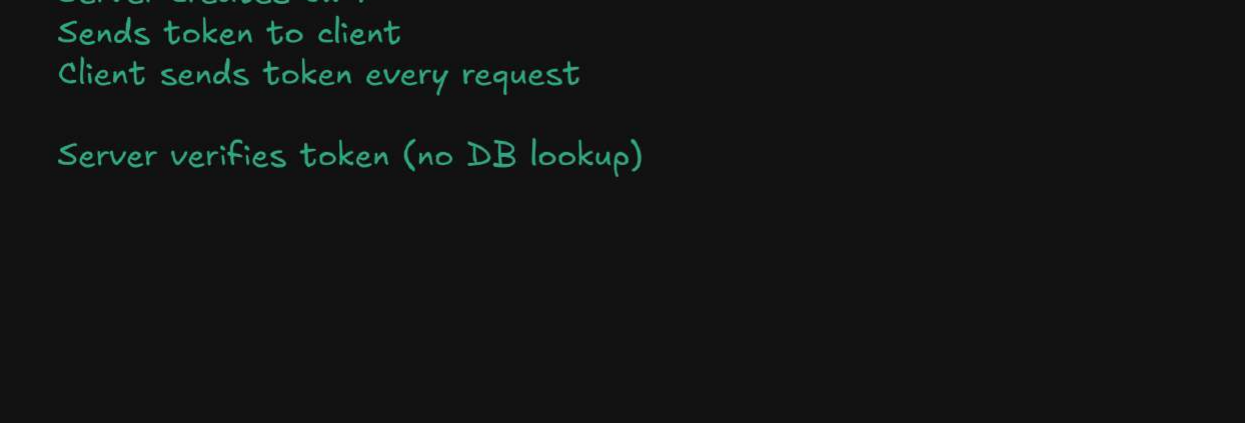


Flow

User logs in
Server creates session
Server stores it (DB/Redis)
Client gets sessionId (cookie)

Every request:
server looks up session

JWT-Based Authentication



Flow

User logs in
Server creates JWT
Sends token to client
Client sends token every request

Server verifies token (no DB lookup)

Feature	Session	JWT
State	Stateful	Stateless
Storage	Server	Client
DB lookup	Every request	Not needed
Scalability	Harder	Easier
Logout	Easy	Hard
Token size	Small	Larger
Security control	Strong	Tricky

Real-World Usage

Use JWT when:

APIs / mobile apps
Microservices
OAuth / SSO
Distributed systems

Use Sessions when:

Traditional web apps
Need strict control
Banking / high security
Admin panels

Common Misconceptions

"JWT replaces sessions"

→ No

"JWT is always better"

→ Depends on use case

Real Production Architecture

Typical modern system:

Access Token → JWT (stateless)
Refresh Token → stored (stateful)

-> Balance of:

performance
security

Does Secret lookup count as db lookup similar to sessions ?

Secret stored in AWS Secrets Manager or HashiCorp Vault

No, JWT verification does not involve a database lookup because the signing key is loaded from a secrets manager once at server startup and cached in memory.
All subsequent token verifications are done locally using this in-memory key, keeping the system stateless and efficient